Break Fast

António Santos - 119139

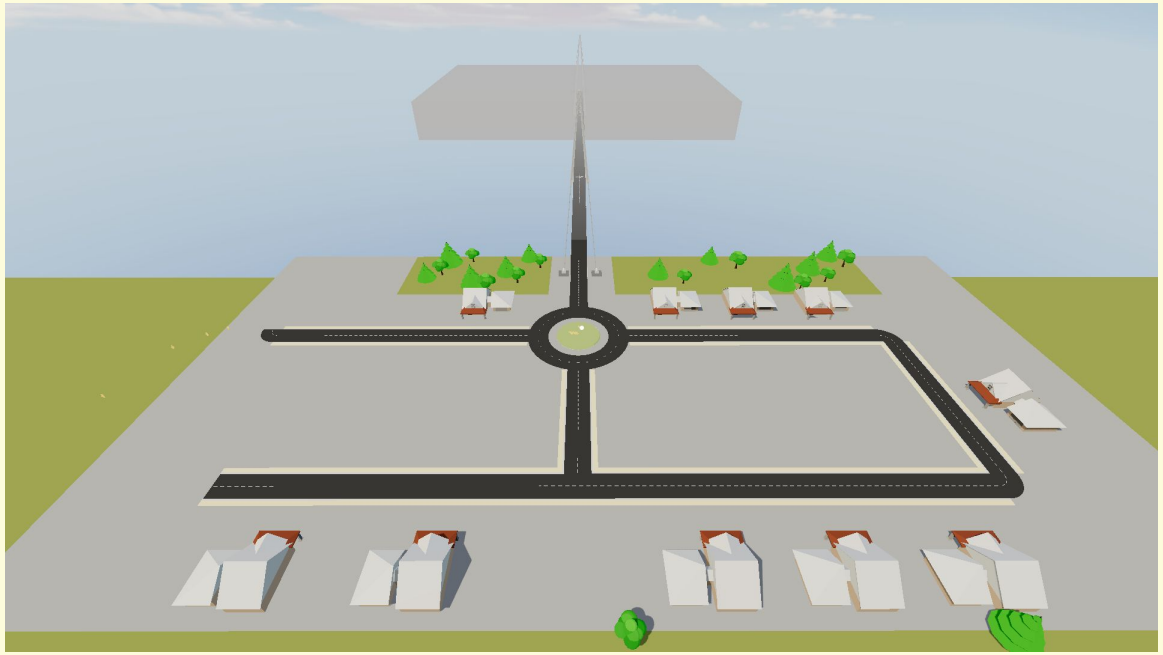
https://github.com/Apmds/break-fast

https://apmds.github.io/break-fast/

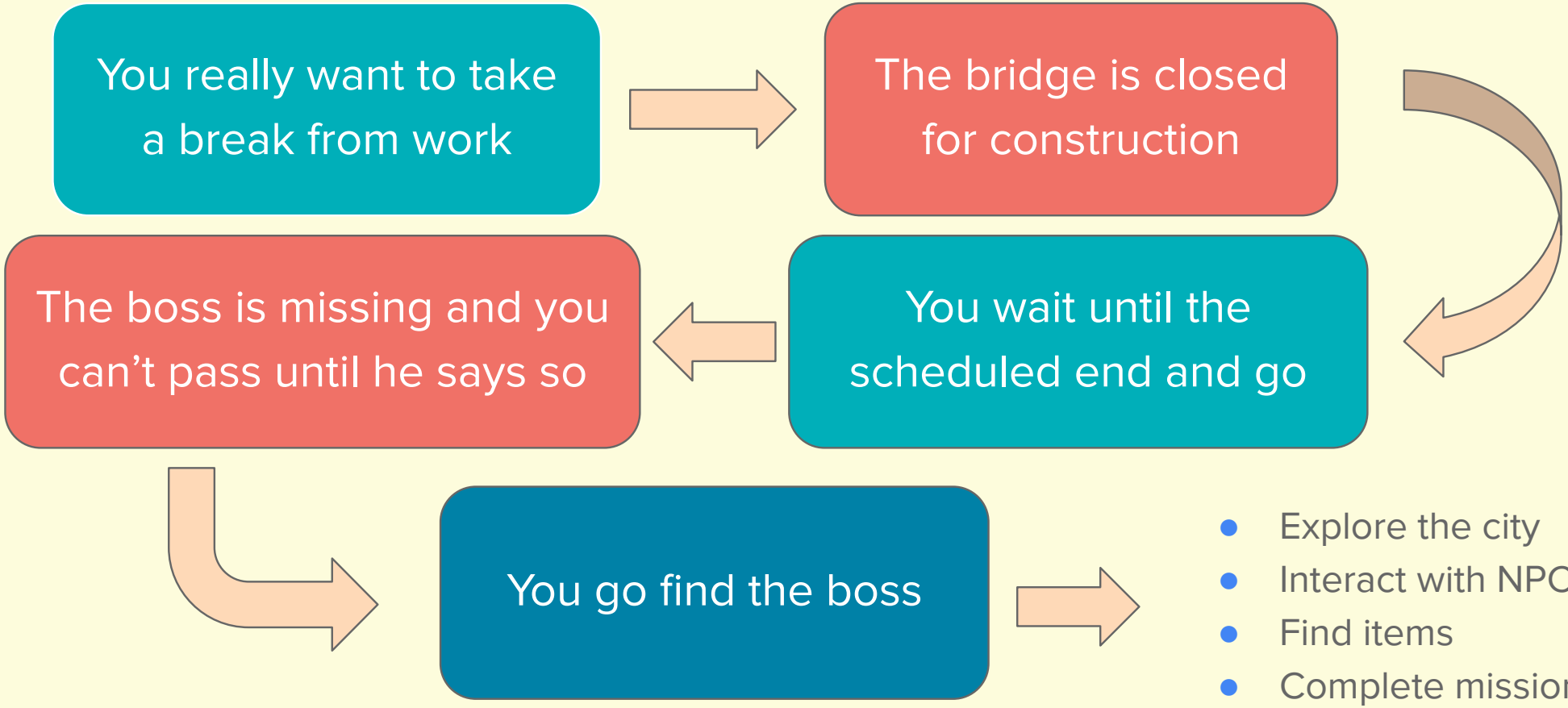
deti universidade de aveiro
departamento de eletrónica, telecomunicações e informática

The Project

A game about going on holiday quickly.

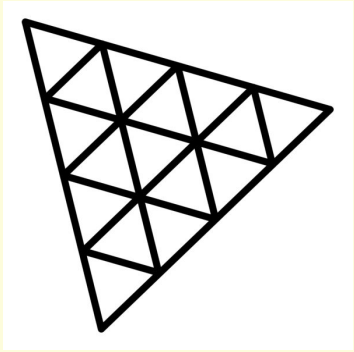


The Project



- Explore the city
- Interact with NPCs
- Find items
- Complete missions

Tools



r184 (most recent)

Addons:

- SkeletonUtils
- GLTFLoader
- stats
- BufferGeometryUtils

cannon-es

0.20.0

(including debugger)

lil-gui

0.20.0

Tools

3D modelling

GitHub Pages

Deployment

ambientCG

FREE STYLIZED .COM

freepik

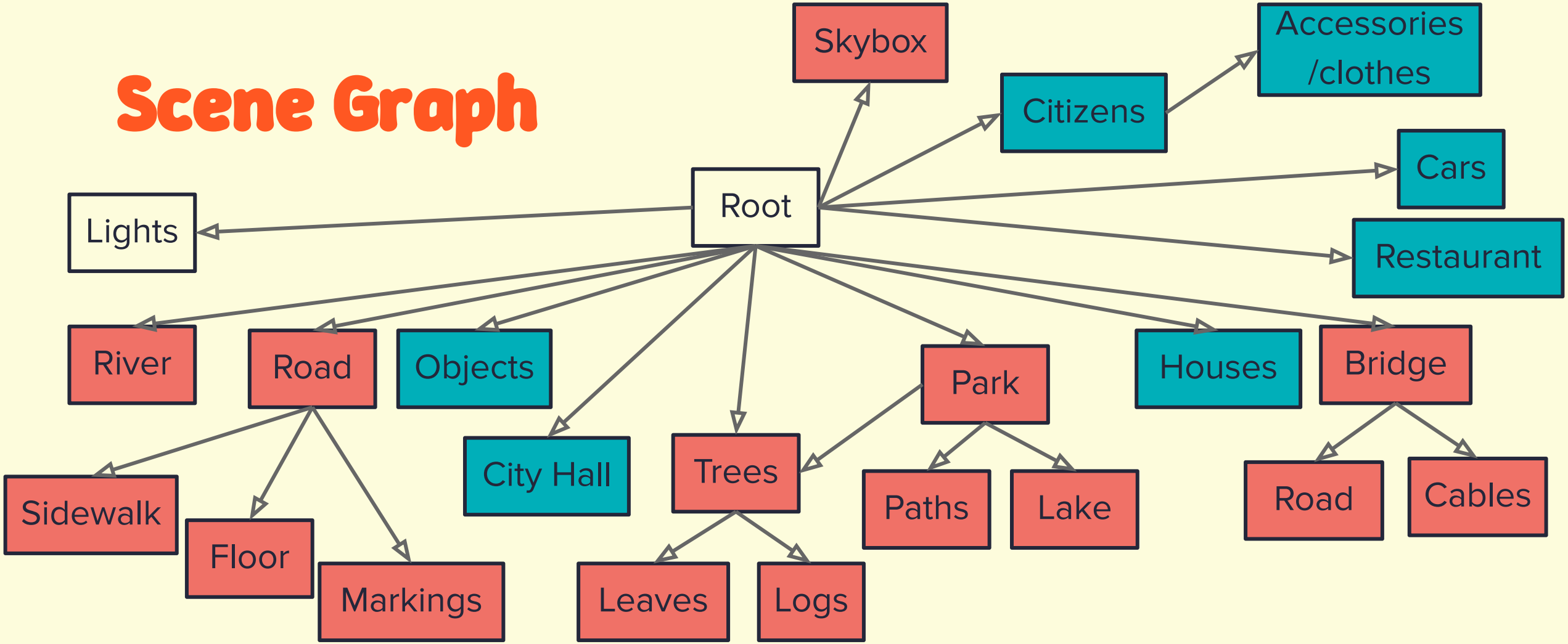
Textures and materials

(some) 3D models

Fonts

Animations

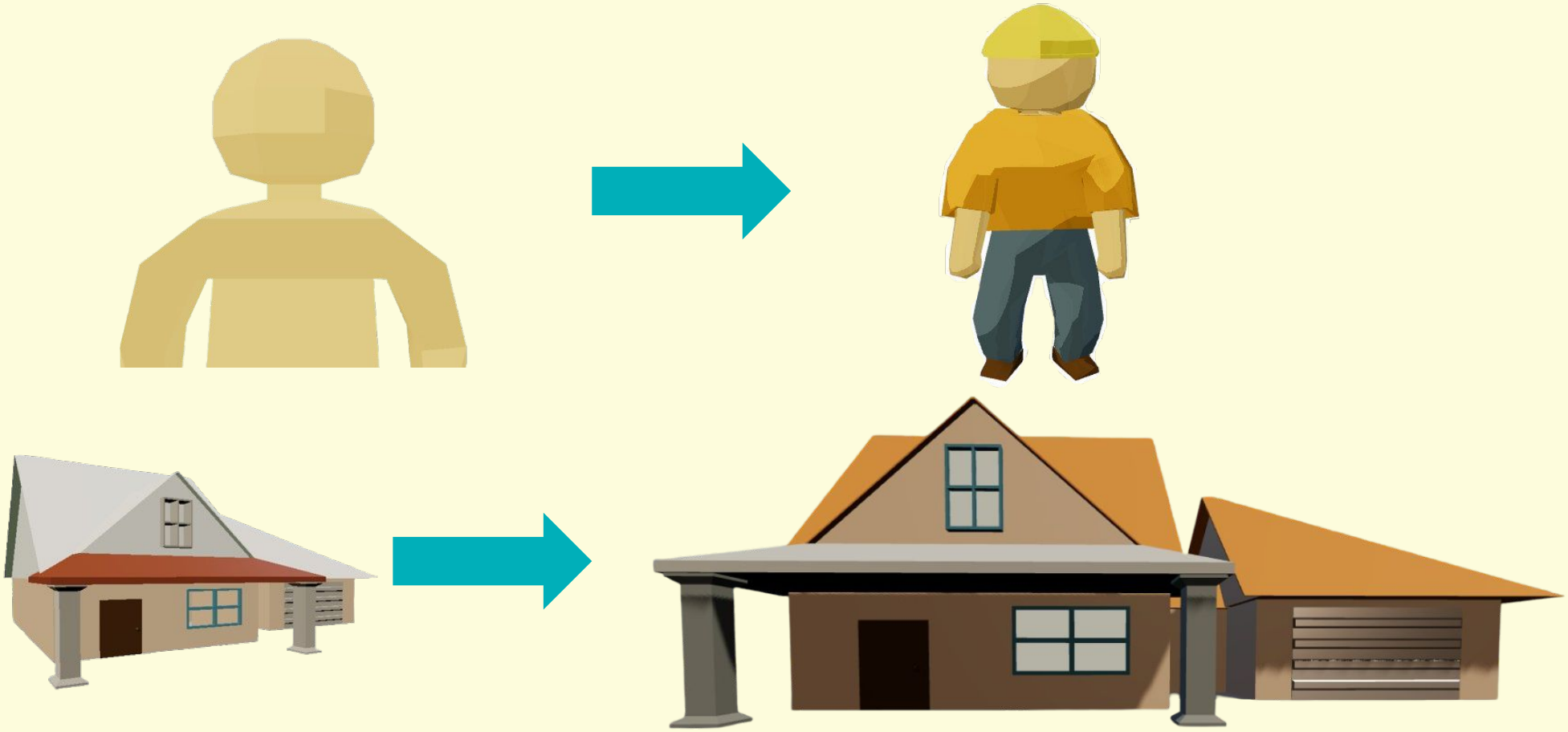
Scene Graph



- Blender models
- Threejs models

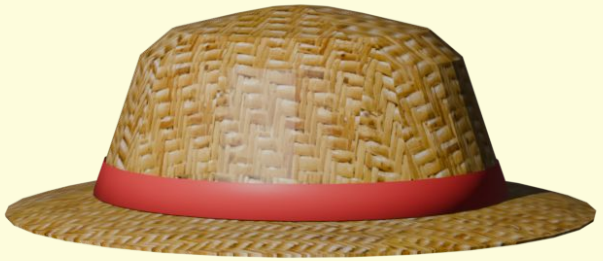
Models

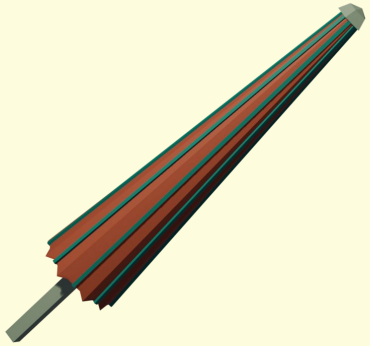
- From both Three.js and Blender



Models

Trees (Threejs)

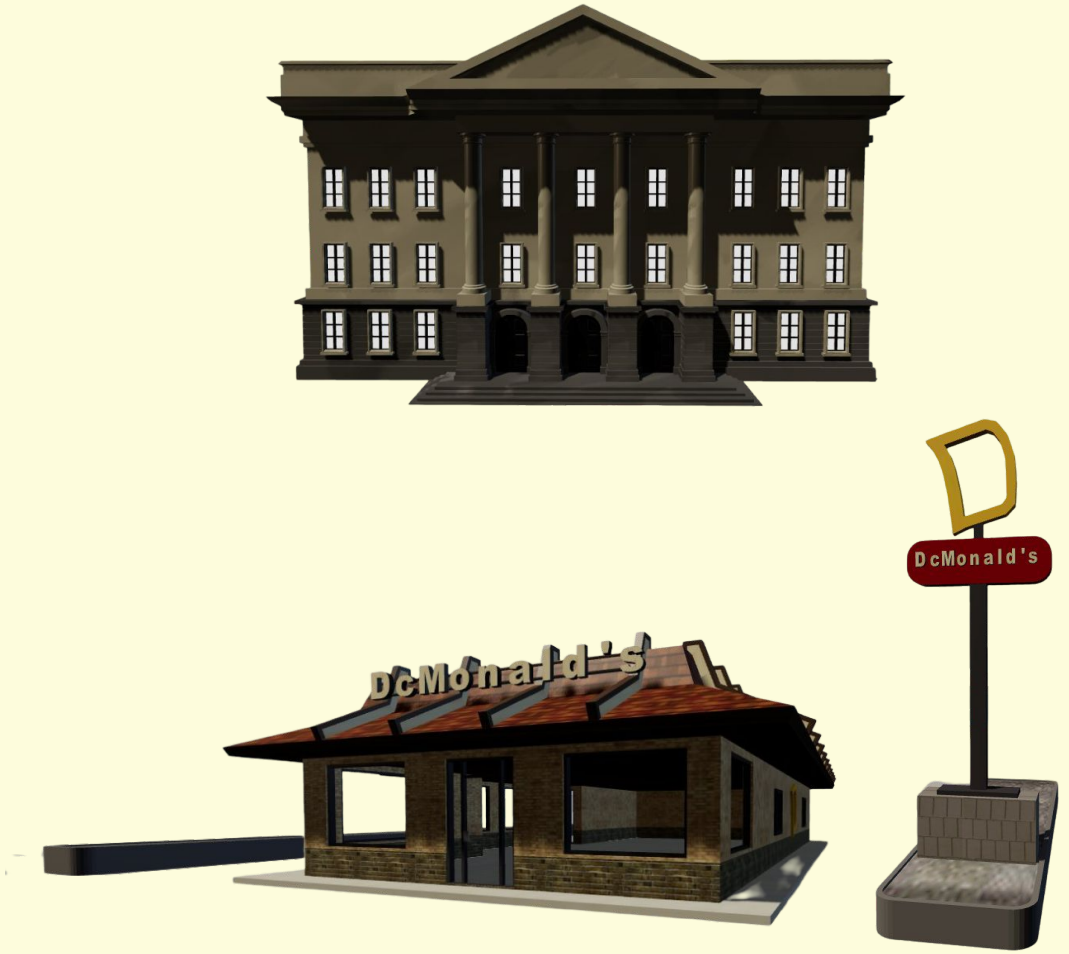




Objects (Blender)

Models

Car (Blender)



Buildings (Blender)

Models

Bridge (Threejs)

Roads/sidewalks (Threejs)

Materials

- Most models use either *MeshToonMaterial* or *MeshStandardMaterial*.
- *MeshToonMaterial* uses *gradientMap* texture.
- *MeshStandardMaterial* uses PBR features (roughness, metalness, normal map, ...).

Citizen (*MeshToonMaterial*)

House (Principaled BSDF / *MeshStandardMaterial*)

Materials

- Object outlines use *ShaderMaterial*.
- Vertex shader extrudes vertices based on normal vector.
- Fragment shader sets outline color.

```
uniform float thickness;

#include <skinning_pars_vertex>

void main() {
    vec3 objectNormal = vec3(normal);
    vec3 transformed = vec3(position);

    // Apply skeletal animations
    #include <skinbase_vertex>
    #include <skinnormal_vertex>
    #include <skinning_vertex>

    // Extrude the mesh
    vec3 pos = transformed + objectNormal * thickness;

    gl_Position = projectionMatrix * modelViewMatrix * vec4(pos, 1.0);
}
```

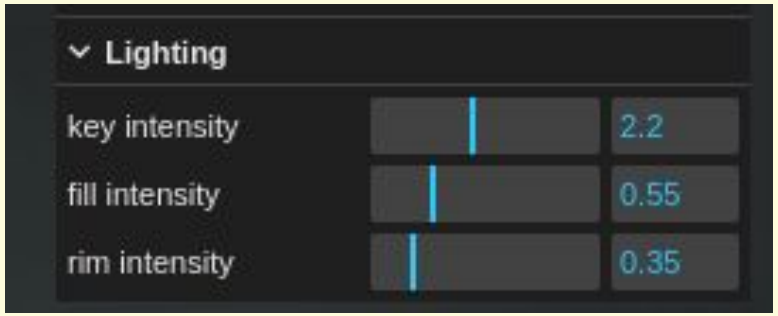
```
uniform vec3 outlineColor;

void main() {
    gl_FragColor = vec4(outlineColor, 1.0);
}
```

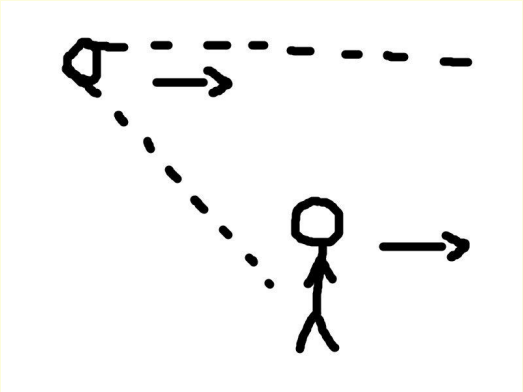
Outline (*ShaderMaterial*)

Illumination

- Scene has 4 light sources: 1 *AmbientLight* and 3 *DirectionalLights*.
- Ambient, fill and rim lights control light level.
- Only the key light provides shadows.
- Key light moves relative to player.



All DirectionalLights

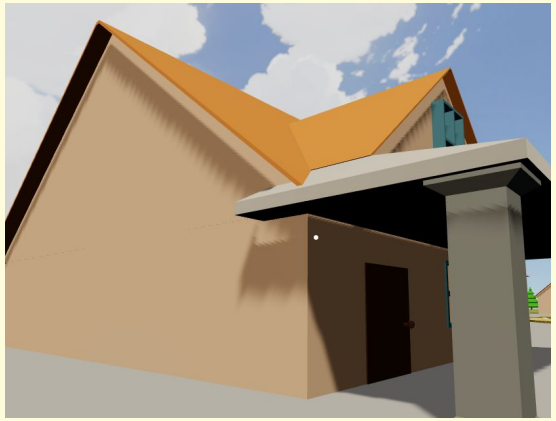


Realistic depiction of keylight movement

```
const keyLight = this.getObject("keyLight");
keyLight.position.copy(new THREE.Vector3().addVectors(this.player.position, this.sunpos));
```

Shadows

Big map + Only 1 light for all shadows = Performance issues

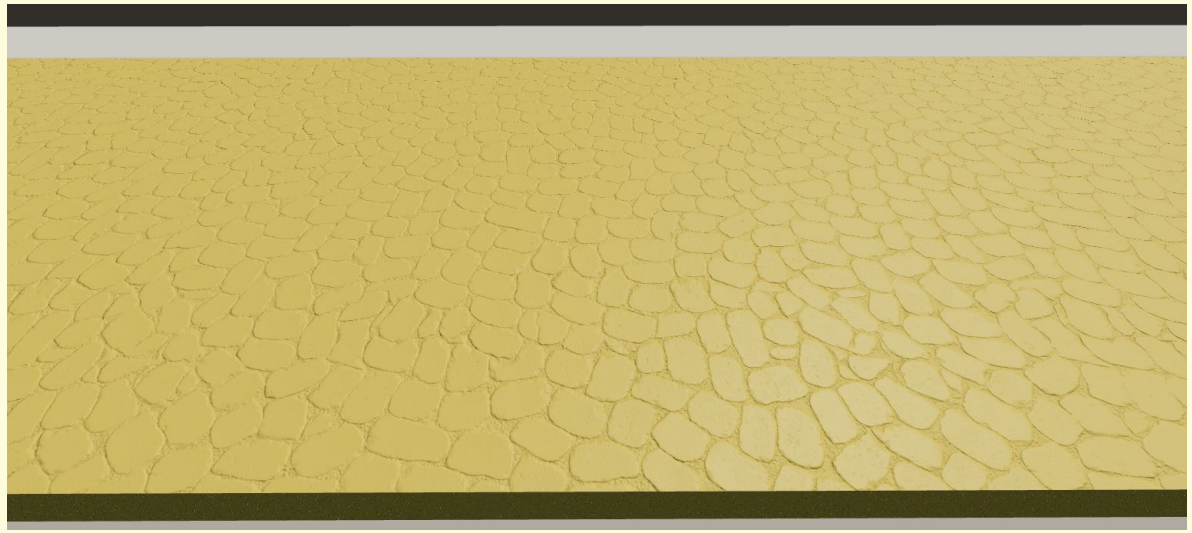


Jagged shadows due to small shadow map size.

Fix: hide some “uglier” shadows

Partial shadows because of small shadow camera frustum.

Textures

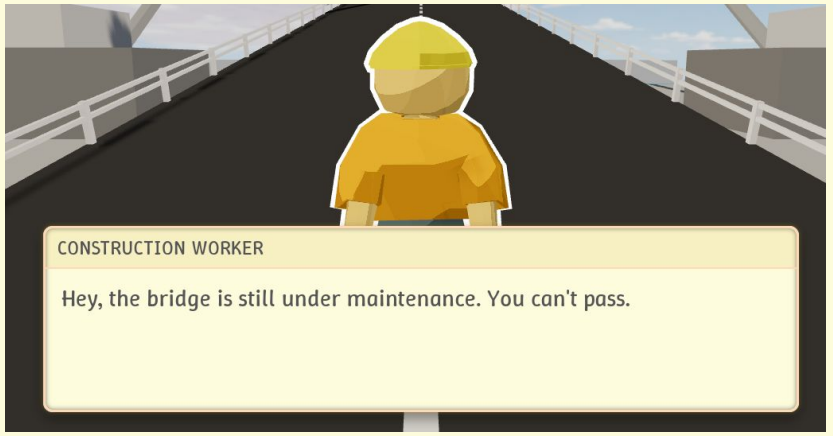


- Normal map
- Roughness map
- Ambient occlusion

- Normal map
- Displacement map
- Roughness map
- Base color texture
- Ambient occlusion

Sound

- Citizens “speak” during conversation with a fast series of grunts.
- Inspired in Animal Crossing.
- Each citizen has a different pitch.



Animation

- Player model was rigged and imported into Mixamo and animations were imported back to Blender.
- Car animation uses noise modifier in Blender to move the frame and rotates wheels on the Y axis.
- All objects of type *WorldObject* have an internal AnimationMixer through which they play actions.

Animation

- *WorldObject*s have the capability to follow paths of many points with position, rotation and speed.
- The vectors of two points are linearly interpolated every frame to set the location of the object based on the speed variable.

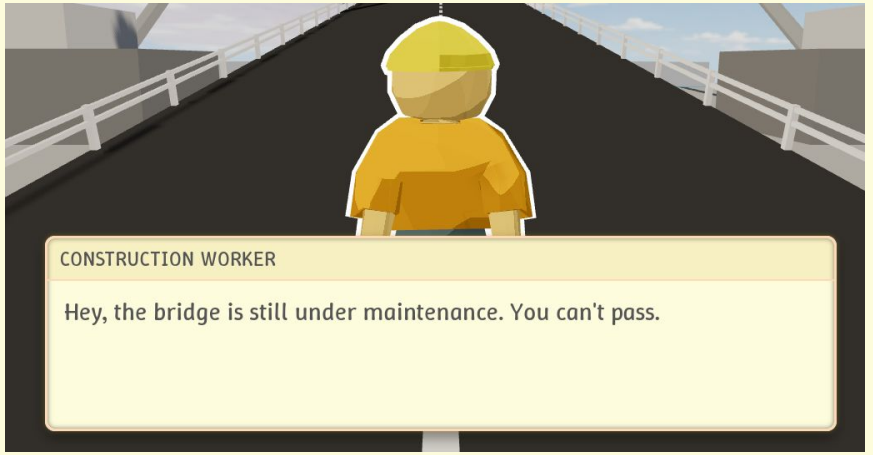
```
citizen4.setPath(new Path()
    .addPoint(new THREE.Vector3(23, 0.4, -393), new THREE.Vector3(0, Math.PI, 0), 10)
    .addPoint(new THREE.Vector3(23, 0.4, -594.4), new THREE.Vector3(0, Math.PI, 0), 0.15)
    .addPoint(new THREE.Vector3(24, 0.4, -603.8), new THREE.Vector3(0, 3*Math.PI/4, 0))
    .addPoint(new THREE.Vector3(31, 0.4, -605), new THREE.Vector3(0, Math.PI/2, 0))
    .addPoint(new THREE.Vector3(159, 0.4, -605), new THREE.Vector3(0, Math.PI/2, 0), 0.15)

    .addPoint(new THREE.Vector3(159, 0.4, -605), new THREE.Vector3(0, 3*Math.PI/2, 0), 10)

    .addPoint(new THREE.Vector3(31, 0.4, -605), new THREE.Vector3(0, 3*Math.PI/2, 0), 0.15)
    .addPoint(new THREE.Vector3(24, 0.4, -603.8), new THREE.Vector3(0, 7*Math.PI/4, 0))
    .addPoint(new THREE.Vector3(23, 0.4, -594.4), new THREE.Vector3(0, 8*Math.PI/4, 0))
    .addPoint(new THREE.Vector3(23, 0.4, -393), new THREE.Vector3(0, 8*Math.PI/4, 0), 0.15)
);
citizen4.followPath(true);
```

Controls

- The game uses a perspective camera and first person controls to better match the real world.
- On PC, the game can be controlled with:
  - **WASD**: Move character
  - **Mouse**: Move camera
  - **Shift**: Run
  - **E**: Interact (with people, items, etc.)
  - **esc**: Unlock mouse
- On mobile, there are on-screen **buttons** and **joysticks** to perform actions.
- Interaction is done through a raycast pointing forwards.

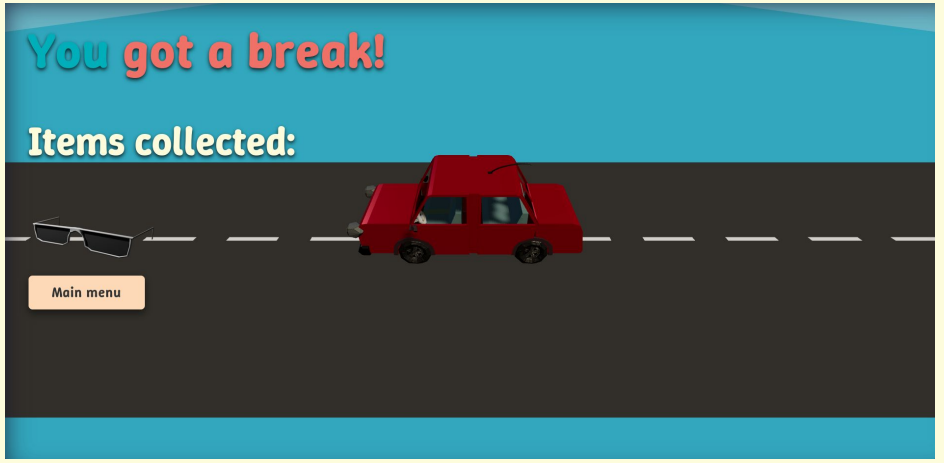


Example of interaction

Menus

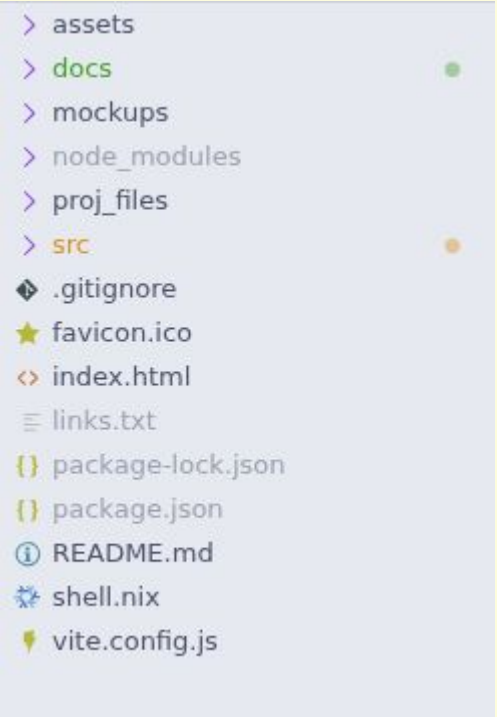

Break Fast
Start Game


Sunglasses
Cool pair of sunglasses given to you from by The Boss. A good item for the holidays.


You got a break!
Items collected:
Main menu

Development

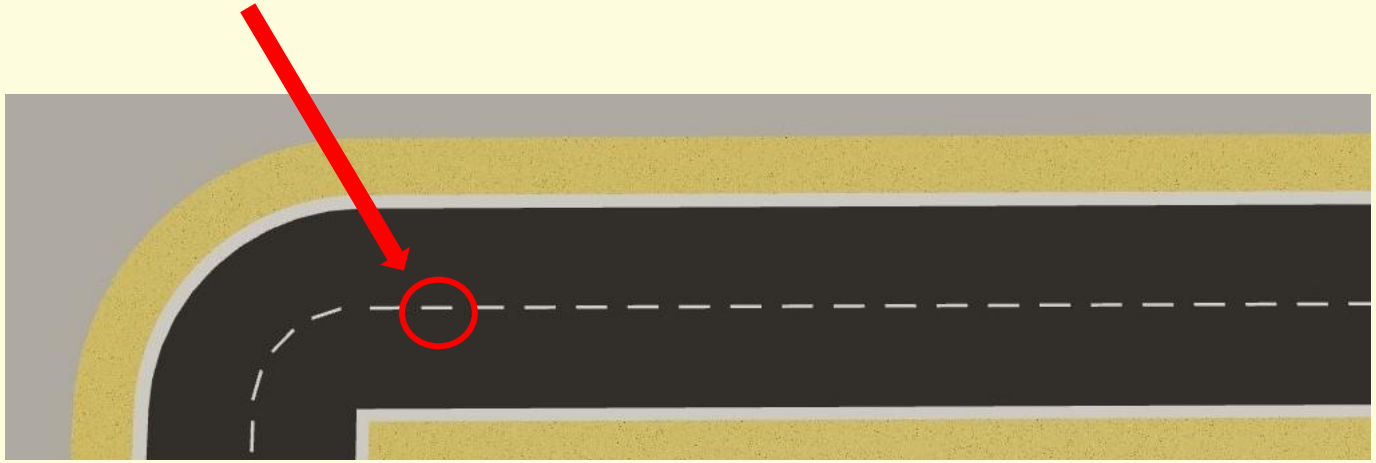
- Files separated into game assets, code and project files.
- Code split into many classes for abstraction and separation.
- Project developed locally using vscode live server.
- Github pages used for deployment.
- Github projects used to manage issues and track progress.



Project files

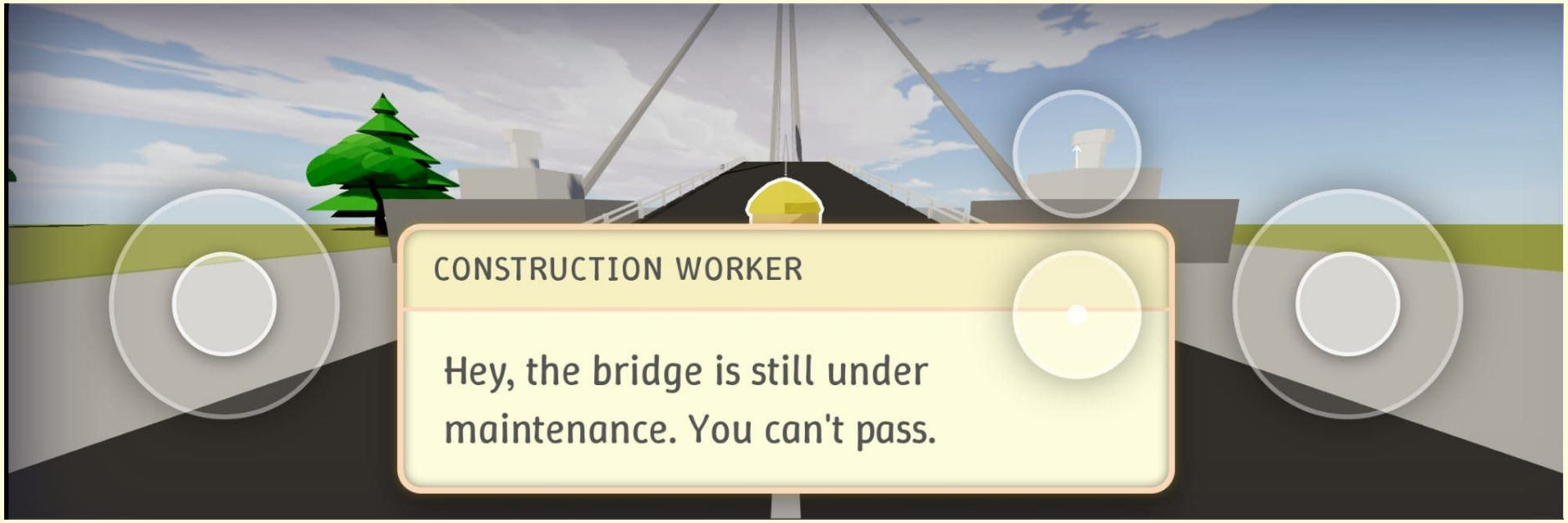
Performance

- Used *InstancedMesh* on very repetitive objects.
- Limited number of light sources (and ones that produce shadows).
- Game meant to run at 60fps in power saving mode, with slight dips.



Performance

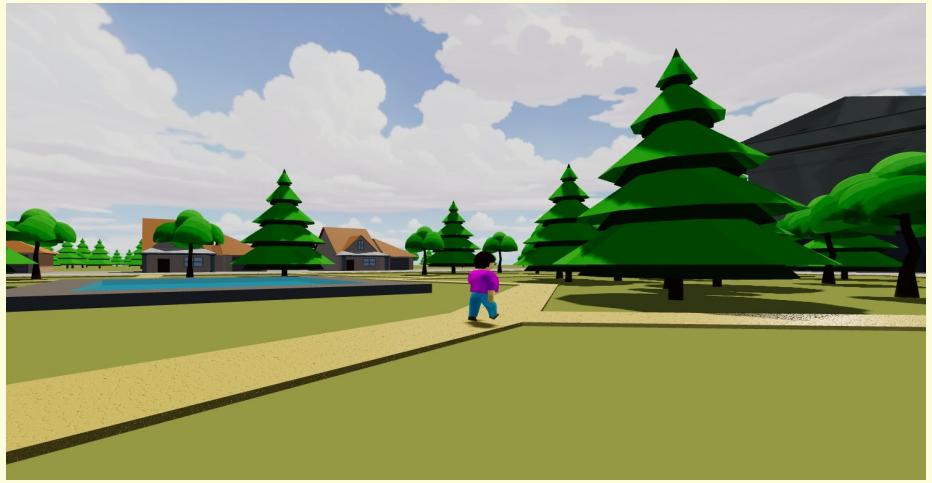
- Game playable on all modern browsers and smartphones (with some issues).



Interface has controls but is not completely responsive.

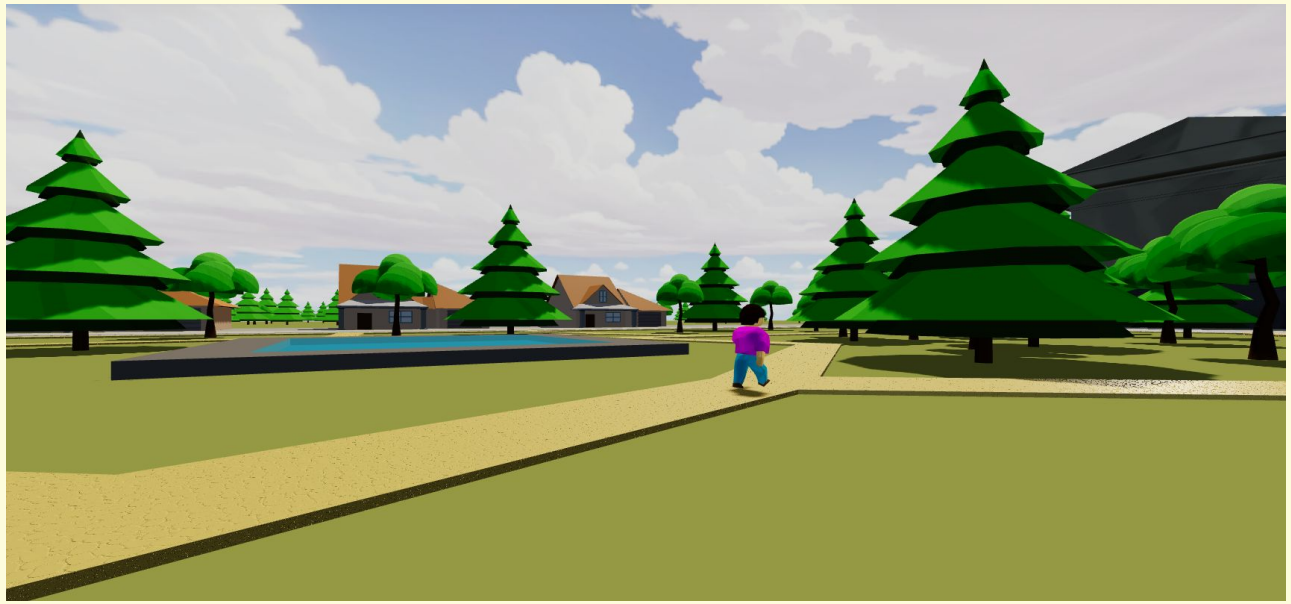
Other Features

- Conversations
- Items
- Pathing



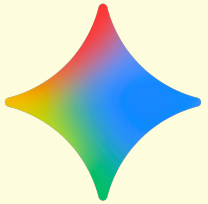
Conclusions

- Scope creep must be kept in check.
- Embrace more external models.



Use of AI

- Google Gemini was used:
  - As a guide for operating Blender.
  - For help creating the CSS.
- GitHub Copilot was used:
  - For help integrating outline shader with bone animations.
  - For help translate camera controls into javascript code.
- Claude:
  - Implemented the mobile controls.
  - Created the lines on the “item acquired” menu.
  - Was used to debug the code.
  - Helped implement the audio system.

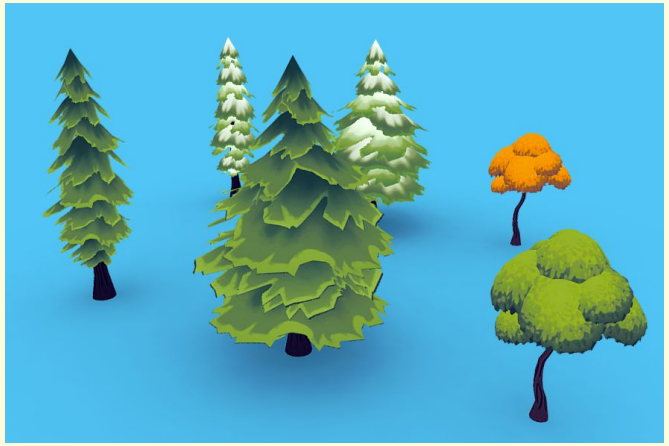


References

- Tutorials:
  - mojoGameDev - Simple Blender Clothing Tutorial That Actually Works!
  - Joey Carlino - Rigging for impatient people - Blender Tutorial
  - Joey Carlino - Character animation for impatient people - Blender Tutorial
  - Blender reference manual - https://docs.blender.org/manual/en/latest/
  - Three.js documentation - https://threejs.org/docs/
  - Learn OpenGL - https://learnopengl.com/
- Models:
  - novusod - McDonalds model (Sketchfab)
  - Dybo - Utrecht Stadhuis/City Hall (Sketchfab)
- Materials/textures:
  - Skybox and Pavement textures.
  - Tire material and Wheel rim texture.
  - Straw texture.

References

- Visual References:



rotblush - Stylized Trees Bundle 01 (Sketchfab)

Reference for the house model

DEMO